Budapesti Műszaki SZC Bláthy Ottó Titusz Informatikai Technikum

1032 Budapest, Bécsi út 134.

Szoftverfejlesztő- és tesztelő képzés

VIZSGAREMEK

Kézműves Webshop alkalmazás fejlesztése Laravelben, Vue.js JavaScript keretrendszer alkalmazásával

Készítette:

- **Neve:** Tóth Balázs
- **E-mail címe:** toth.balazs@blathy.info
- **Neve:** Krupa Levente
- **E-mail címe:** krupa.levente@blathy.info
- **Neve:** Chrenkó Balázs
- **E-mail címe:** chrenko.balazs@blathy.info

VIZSGAREMEK ADATLAP

A Vizsgaremek témája: Az adattárolási és kezelési feladatokat is megvalósító program egy kézműves termékeket forgalmazó webalkalmazás, amely a termékkatalógus nyilvántartását, a vásárlói kosár kezelését és a leadott rendelések állapotának nyomon követését valósítja meg. A normál felhasználó a kínálat megtekintésére, kosárba helyezésére és vásárlásra használja, az adminisztrátor módosításokat és termékek hozzáadását is végezheti, valamint frissíti a rendelések státuszait.

A Vizsgaremek címe: Kézműves Webshop alkalmazás fejlesztése Laravelben, Vue.js JavaScript keretrendszer alkalmazásával

Dátum: 2026.04.27.

EREDETISÉG NYILATKOZAT

Alulírott tanulók kijelentjük, hogy a vizsgaremek saját munkánk eredménye, a felhasznált szakirodalmat és eszközöket azonosíthatóan közöltem. Az elkészült vizsgaremek részét képező anyagokat az intézmény archiválhatja és felhasználhatja.

Dátum: 2026.04.27.

Tartalomjegyzék

Budapesti Műszaki SZC Bláthy Ottó Titusz Informatikai Technikum....1.
1032 Budapest, Bécsi út 134....1.
VIZSGAREMEK....1.
Kézműves Webshop alkalmazás fejlesztése Laravelben, Vue.js JavaScript keretrendszer alkalmazásával....1.
VIZSGAREMEK ADATLAP....2.
EREDETISÉG NYILATKOZAT....3.
1 A projekt ismertetése....5.
1.1 A technológiai verem (Tech Stack) kiválasztásának indoklása....6.
2 Felhasználói dokumentáció....7.
2.1 Felületek ismertetése....7.
2.2 Funkciók ismertetése és használati instrukciók....9.
2.3 Képernyőméretek és reszponzív megjelenés....10.
2.4 Hibajelzések és rendszerüzenetek....11.
2.5 Részletes felhasználói folyamatok (Workflow)....11.
2.6 Kijelentkezés a rendszerből....11.
3 Fejlesztői dokumentáció....12.
3.1 Munkamegosztás....12.
3.1.1 Szerepkörök és felelősségi körök....12.
3.2 Telepítési dokumentáció....13.
3.2.1 Adatbázis előkészítése (XAMPP)....13.
3.2.2 Backend telepítése (Laravel)....14.
3.2.3 Teszt felhasználók....14.
3.2.4 Frontend telepítése (Vue.js)....14.
3.2.5 Adatbiztonság és hitelesítési mechanizmusok....15.
3.3 Adatbázis dokumentáció....15.
3.3.1 Diagram....15.
3.3.2 Mezők részletes ismertetése....16.
3.3.3 Kapcsolatok indoklása, ismertetése....19.
3.4 Komponens/struktúra dokumentáció....19.
3.5 API dokumentáció....20.
API végpontok részletezése....20.
3.6 Tesztelési dokumentáció....22.
4 Továbbfejlesztési lehetőségek....29.
5 Összegzés....29.
6 Források....30.

1 A projekt ismertetése

A Kézműves Webshop alkalmazás a Szoftverfejlesztő és tesztelő képzés vizsgafeladataként készült, és a képzés során elsajátított tudást hivatott bemutatni egy közös munka keretében. Az adattárolási és kezelési feladatokat is megvalósító program egy modern e-kereskedelmi platform, amely az elérhető árukészletet és a beérkező vásárlói rendeléseket tartja nyilván.

A rendszer három különböző felhasználói szintet és jogosultsági kört határoz meg. A normál felhasználó (látogató) átlátja a webshop kínálatát, használhatja a beépített keresőt és a kategóriaszűrőket, valamint összeállíthatja a kosara tartalmát. Számára az elsődleges cél az információszerzés és a termékek megismerése. A vásárlás véglegesítéséhez és a kosár tartalmának megrendeléséhez a rendszer felhasználói regisztrációt és bejelentkezést követel meg.

A regisztrált vásárló hitelesítés után hozzáférést kap a rendelés leadásának funkciójához. Mivel a program nem csak az aktuális nyomon követésre képes, hanem azt is megmutatja, hogy a korábbi tranzakciók pontosan milyen tételeket tartalmaztak, a vásárló a saját profilimportjában a múltbeli rendeléseit is visszakövetheti. A rendelés leadását követően az adminisztrátor által végzett státuszváltozások (például feldolgozás alatt, kiszállítva) valós időben frissülnek a vásárló profiloldalán.

A webshop adatlapját szerkeszteni csak az adminisztrátor tudja: módosíthatja a termék nevét, árát, leírását és kategóriáját. Új termék beszerzésekor és a webshopba történő feltöltésekor szintén csak ő tud eljárni, ekkor természetesen minden mező megadható, valamint a termékhez tartozó képfájl feltöltése is a szerverre történik. Továbbá az adminisztrátor felel a beérkezett vásárlói rendelések áttekintéséért, a teljesítési folyamat adminisztrálásáért, a státuszok módosításáért és az esetleges téves rendelések törléséért.

A feladat megvalósításához backend oldalon a Laravel nyílt forráskódú PHP keretrendszert alkalmaztuk MySQL relációs adatbázis-kezelő rendszer használatával, a frontend pedig Vue.js JavaScript keretrendszerben készült.

1.1 A technológiai verem (Tech Stack) kiválasztásának indoklása

A modern webfejlesztési trendeknek megfelelően a rendszert szétválasztott (decoupled) architektúrában terveztük meg. A backend és a frontend nem egyetlen monolitikus alkalmazásként fut, hanem REST API végpontokon keresztül, JSON formátumban kommunikál egymással. Ennek a megközelítésnek az elsődleges előnye a skálázhatóság: a Vue.js alapú Single Page Application (SPA) frontend gondoskodik a gyors, újratöltés nélküli oldalváltásokról, ami drasztikusan javítja a felhasználói élményt (UX). Eközben a Laravel backend kizárólag az üzleti logikára, az adatbázis-műveletekre és a hitelesítésre fókuszál. Ez a struktúra a jövőben lehetővé tenné akár egy natív mobilalkalmazás (pl. Android/iOS) fejlesztését is, hiszen az API végpontok már készen állnak a kiszolgálásra.

2 Felhasználói dokumentáció

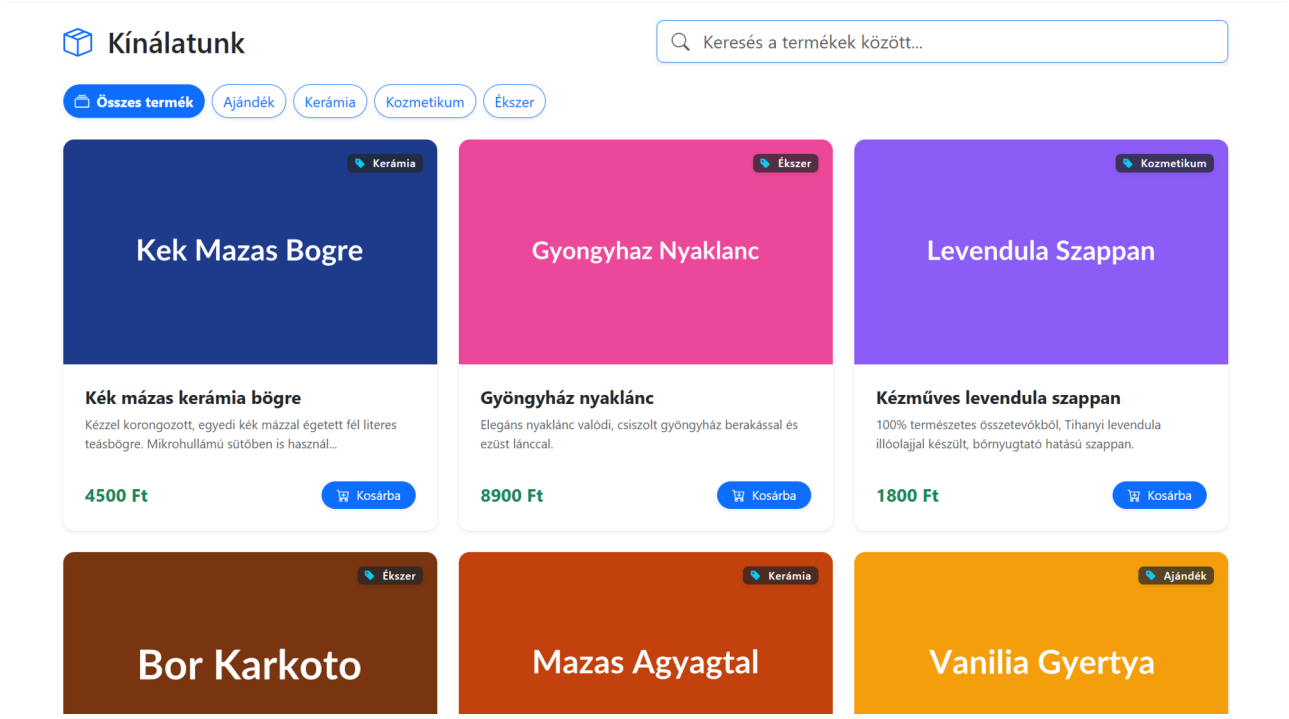
A felhasználói dokumentáció célja, hogy segítse a látogatókat és az adminisztrátorokat a rendszer hatékony használatában. Bemutatja a grafikus felület elemeit, a navigációt és a különböző funkciók elérésének módját.

2.1 Felületek ismertetése

A webshop felülete letisztult és modern, a navigációt a képernyő felső részén elhelyezett menüsor segíti. A kezdőoldal egy látványos üdvözlőképernyővel (Hero szekció) fogadja a látogatót.

1. Ábra: A kezdőoldal (Hero szekció) megjelenése

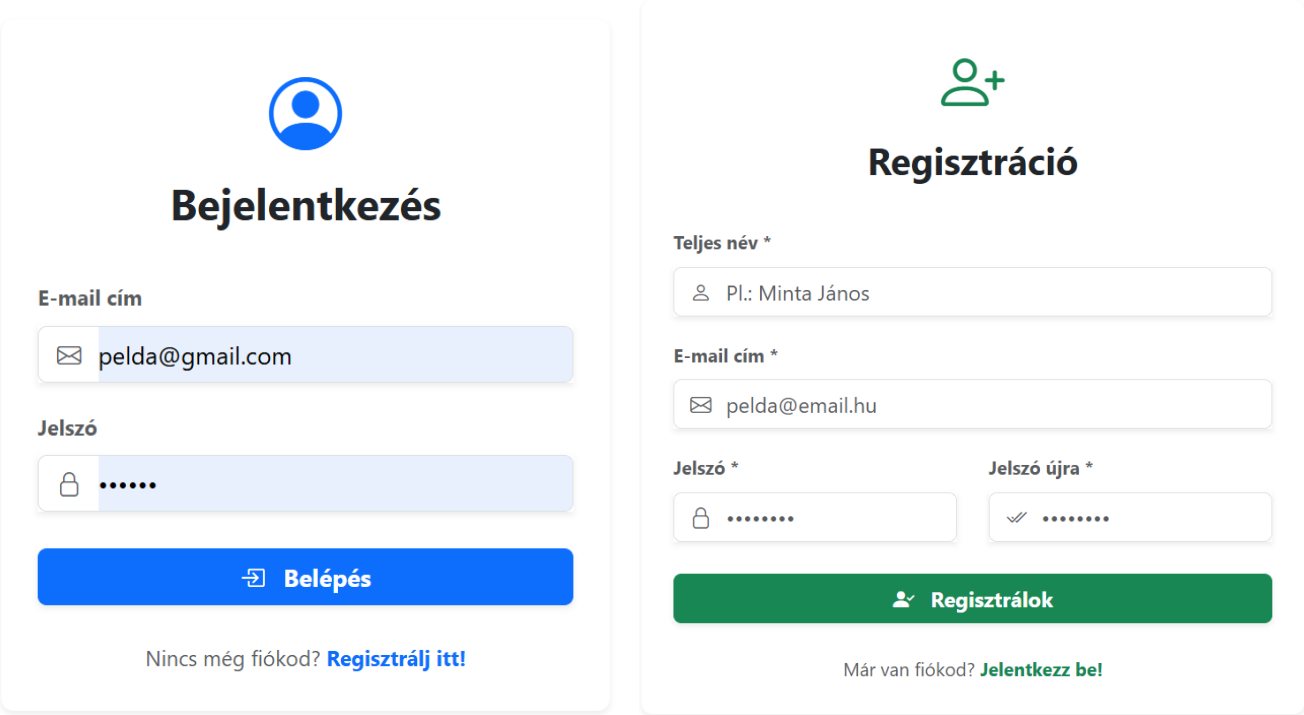
A "Termékek" menüpont alatt található a katalógus, ahol az árucikkek kártyás elrendezésben láthatók. Minden kártyán szerepel a termék fotója, neve, ára és kategóriája.



2. Ábra: A termékkatalógus kártyákkal és szűrőkkel

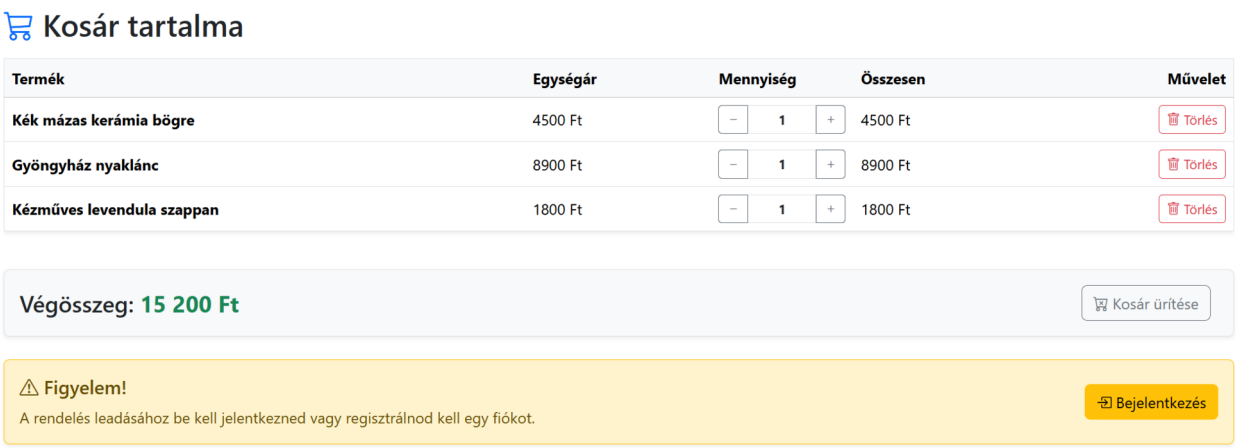
2.2 Funkciók ismertetése és használati instrukciók

A rendszer alapvető funkciói a termékkeresés, a kosárba helyezés és a rendelésleadás. A bejelentkezési felületen a regisztrált felhasználók férhetnek hozzá profiljukhoz, míg az új látogatók a regisztrációs űrlapon hozhatnak létre fiókot.



3. Ábra: Bejelentkezési és regisztrációs űrlapok

A "Kosár" menüpontban a kiválasztott tételek darabszáma módosítható vagy törölhető. A rendelés véglegesítése után a rendszer rögzíti a vásárlást.



4. Ábra: A Kosár felület összesítése

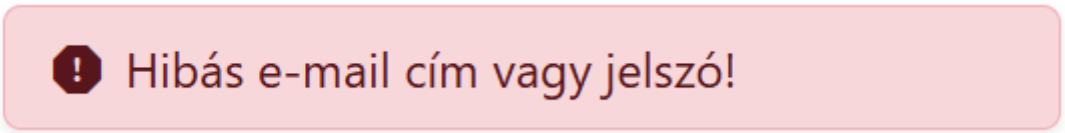
2.3 Képernyőméretek és reszponzív megjelenés

Az alkalmazás Bootstrap 5 keretrendszerrel készült, így minden eszközön (mobil, tablet, PC) optimálisan jelenik meg. Mobilnézetben a menüpontok egy lenyíló "hamburger" menübe kerülnek.

5. Ábra: Mobilnézet és hamburger menü

2.4 Hibajelzések és rendszerüzenetek

A program vizuális üzenetekkel segíti a felhasználót. Sikeres művelet esetén zöld, hiba (például hiányos adatok) esetén piros értesítések jelennek meg.

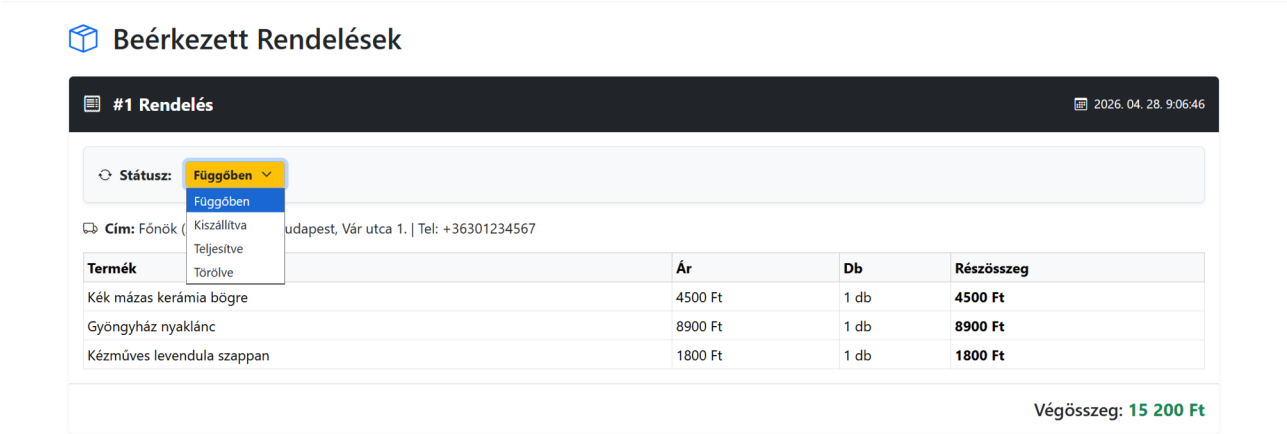


6. Ábra: Validációs hibaüzenetek példája

2.5 Részletes felhasználói folyamatok (Workflow)

Vásárlói folyamat: A termék kiválasztása után a kosárba helyezés, majd a bejelentkezés következik. Ezt követően a rendelés leadása gombbal véglegesíthető a folyamat, melyről a profiloldalon kapunk visszajelzést.

Adminisztrátori folyamat: Az adminisztrátor a termékkezelő felületen tud új termékeket felvinni, képet feltölteni, vagy a meglévő rendelések státuszát frissíteni.



7. Ábra: Adminisztrátori rendeléskezelő felület

2.6 Kijelentkezés a rendszerből

A munkamenet a navigációs sávban található "Kijelentkezés" gombbal zárható le, amely törli a tokeneket és visszairányít a kezdőoldalra.

3 Fejlesztői dokumentáció

A fejlesztői dokumentáció célja, hogy részletes technikai áttekintést nyújtson a Kézműves Webshop szoftverarchitektúrájáról, az alkalmazott technológiákról és a kód szintű felépítésről. Ez a fejezet bemutatja azokat a háttérfolyamatokat, amelyek a felhasználói felület zavartalan működését biztosítják.

3.1 Munkamegosztás

A Kézműves Webshop projekt fejlesztése egy háromfős csapat keretein belül valósult meg. A feladatok elosztása a projekt kezdetétől fogva egyértelműen meghatározásra került, figyelembe véve a csapattagok egyéni kompetenciáit.

3.1.1 Szerepkörök és felelősségi körök

- **Tóth Balázs:** A projekt frontend oldalának teljes körű kialakításáért felelt. Feladata volt a Vue.js komponensek létrehozása, a reszponzív dizájn finomhangolása és az adminisztrátori szerkesztő felületek megvalósítása.
- **Krupa Levente:** A backend fejlesztésért felelt. Létrehozta a Laravel alapstruktúrát, a modelleket, a migrációkat és a RESTful API végpontokat. Megvalósította a Sanctum alapú hitelesítést és a képfeltöltési logikát.
- **Chrenkó Balázs:** Feladata a két rendszer integrációja, az Axios hívások bekötése és a komplex kosárlogika megvalósítása volt. Felelt a manuális tesztelésért, a Postman ellenőrzésekért és a dokumentáció összeállításáért.

3.2 Telepítési dokumentáció

A rendszer telepítése előtt győződjünk meg arról, hogy a környezet megfelel az alábbi előfeltételeknek: **PHP 8.2+** (és Composer), **Node.js** (LTS verzió), valamint a **XAMPP** csomag (Apache és MySQL/MariaDB modulok).

3.2.1 Adatbázis előkészítése (XAMPP)

A szoftver futtatásához szükség van egy helyi adatbázis-szerverre.

1. Indítsuk el a **XAMPP Control Panel** alkalmazást, és aktiváljuk az **Apache** és **MySQL** modulokat.
2. Kattintsunk a MySQL sorában található **Admin** gombra, vagy navigáljunk a böngészőben a http://localhost/phpmyadmin címre.
3. A felső menüben válasszuk az **Adatbázisok** fület, vagy a bal oldali sávban az **Új** lehetőséget.
4. Az adatbázis neve mezőbe írjuk be: **kezmuves_db**.
5. Az illesztésnél (Collation) válasszuk az utf8mb4_unicode_ci kódolást, majd kattintsunk a **Létrehozás** gombra.

3.2.2 Backend telepítése (Laravel)

1. **Függőségek telepítése:** Adjuk ki a composer install parancsot a projekt gyökérmappájában.
2. **Környezeti fájl létrehozása:** Másoljuk le az alapértelmezett fájlt: cp .env.example .env.
3. **Adatbázis konfigurálása:** Nyissuk meg a .env fájlt, és ellenőrizzük az alábbi beállításokat:
    - DB_DATABASE=kezmuves_db
    - DB_USERNAME=root
    - DB_PASSWORD= (alapértelmezett XAMPP telepítés esetén üres)
4. **App Key generálása:** Futtassuk a php artisan key:generate parancsot.
5. **Migráció és Seeding:** Futtassuk a php artisan migrate --seed parancsot. Ez felépíti a táblákat és feltölti a rendszert a tesztadatokkal és felhasználókkal.
6. **Tárhely linkelése:** A termékfotók eléréséhez futtassuk a php artisan storage:link parancsot.
7. **Szerver indítása:** Indítsuk el a backend kiszolgálót a php artisan serve paranccsal.

3.2.3 Teszt felhasználók

A sikeres adatbázis-feltöltés (--seed) után a rendszer az alábbi hitelesítő adatokkal tesztelhető:

- **Adminisztrátor:**
    - **E-mail:** admin@webshop.hu
    - **Jelszó:** jelszo123
- **Teszt Vásárló (User):**
    - **E-mail:** vevo@webshop.hu
    - **Jelszó:** jelszo123

3.2.4 Frontend telepítése (Vue.js)

1. **Csomagok telepítése:** Navigáljunk a frontend mappájába, és futtassuk az npm install parancsot.
2. **Alkalmazás indítása:** Az npm run dev parancs elindítja a fejlesztői környezetet.

3.2.5 Adatbiztonság és hitelesítési mechanizmusok

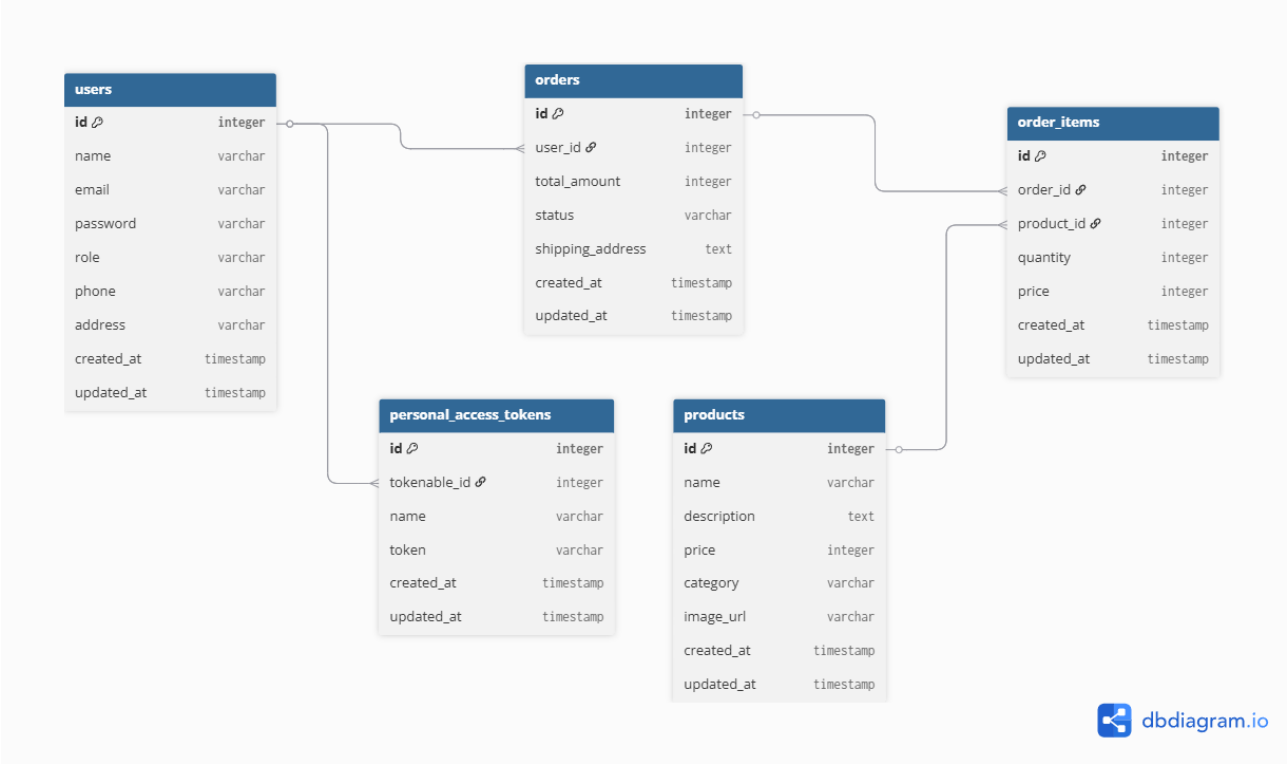
Egy e-kereskedelmi platformnál kritikus szempont a felhasználói adatok védelme. A rendszer hitelesítési rétegét a Laravel Sanctum csomagja biztosítja, amely könnyűsúlyú, token-alapú (API token) autentikációt tesz lehetővé a Vue.js kliens számára. Bejelentkezéskor a szerver egy egyedi, titkosított tokent generál, amelyet a kliens eltárol, és minden további védett API kérés (pl. rendelés leadása) fejlécében (Authorization: Bearer Token) továbbít a szerver felé. A felhasználók jelszavainak tárolása szigorúan titkosítva, a modern iparági szabványnak számító Bcrypt hash algoritmus használatával történik az adatbázisban. Ez garantálja, hogy egy esetleges adatbázis-szivárgás esetén is visszafejthetetlenek maradnak a vásárlói jelszavak.

3.3 Adatbázis dokumentáció

A rendszer adatbázisa öt fő entitásra épül, biztosítva a terméknyilvántartást és a rendelések kezelését.

3.3.1 Diagram

Az alábbi Entity-Relationship (ER) diagram vizuálisan szemlélteti az adatbázis felépítését.



8. Ábra: Adatbázis diagram

3.3.2 Mezők részletes ismertetése

A rendszer stabilitása szempontjából kulcsfontosságú a táblák és a bennük tárolt adatok típusának pontos meghatározása. Az alábbiakban részletezzük a Felhasználók (users), a Hitelesítési tokenek, a Termékek (products), a Rendelések (orders) és a Rendelési tételek (order_items) táblák szerkezetét.

Felhasználók (users) tábla szerkezete

Mező	Típus	Leírás
id	bigint, PK	Egyedi felhasználói azonosító, automatikusan növekvő érték.
name	varchar	A felhasználó teljes neve.
email	varchar, unique	Bejelentkezési e-mail cím, amelynek egyedinek kell lennie.
password	varchar	Biztonságosan titkosított (Bcrypt hash) jelszó.
role	varchar	Jogosultsági kör ('admin' vagy 'user').
phone	varchar	A felhasználó telefonos elérhetősége a kiszállításhoz.
address	varchar	A vásárló szállítási és számlázási címe.

Hitelesítési tokenek (personal_access_tokens) tábla szerkezete

Mező	Típus	Leírás
id	bigint, PK	A token egyedi azonosítója.
tokenable_id	foreignId	Hivatkozás a users tábla adott felhasználójára.
name	varchar	A token megnevezése (pl. mobil vagy asztali).
token	varchar, unique	Maga a titkosított hozzáférési token érték.
created_at	timestamp	A token létrehozásának időpontja.

Termékek (products) tábla szerkezete

Mező	Típus	Leírás
id	bigint, PK	A termék egyedi azonosítója.
name	varchar	A termék pontos megnevezése.
description	text	A termék részletes leírása, anyaga, méretei.
price	integer	A termék bruttó fogyasztói ára (HUF).
category	varchar	A termék kategóriája a szűréshez (pl. Ékszer).
image_url	varchar	A szerverre feltöltött termékfotó elérési útja.

Rendelések (orders) tábla szerkezete

Mező	Típus	Leírás
id	bigint, PK	A rendelés egyedi azonosító száma.
user_id	foreignId	Hivatkozás arra a felhasználóra, aki leadta a rendelést.
total_amount	integer	A rendelés bruttó végösszege forintban.
status	varchar	A rendelés állapota (pl. Függőben, Kiszállítva).
shipping_address	text	A rögzített pontos szállítási cím.

Rendelési tételek (order_items) tábla szerkezete

Mező	Típus	Leírás
id	bigint, PK	A tétel egyedi azonosítója.
order_id	foreignId	Hivatkozás a fő rendelésre (orders tábla).
product_id	foreignId	Hivatkozás a megvásárolt termékre (products tábla).
quantity	integer	A termékből megrendelt darabszám.
price	integer	Az egységár a megrendelés pillanatában.

3.3.3 Kapcsolatok indoklása, ismertetése

A rendszerben egy-a-többhöz (1:N) kapcsolatok állnak fenn a felhasználók és rendelések, valamint a rendelések és tételek között. Ez biztosítja, hogy minden adat visszakövethető és konzisztens maradjon. Egy vásárló több rendelést is leadhat, de minden rendelés egyetlen vásárlóhoz tartozik. A rendelések és tételek közötti kapcsolat lehetővé teszi, hogy egy rendelés több különböző terméket tartalmazzon.

3.4 Komponens/struktúra dokumentáció

A rendszer architektúrája két rétegre oszlik, amelyek REST API-n keresztül kommunikálnak. A backend Laravel alapú, a frontend pedig Vue 3 (SPA) alapú. Ez a modern, szétválasztott felépítés biztosítja a rendszer magas szintű skálázhatóságát és a komponensek könnyű karbantarthatóságát.

Adatbázis tranzakciók (Database Transactions) A rendelés leadásának folyamata egy komplex, több lépésből álló adatbázis-művelet, hiszen egyszerre kell rögzíteni a fő rendelési adatokat (orders tábla) és a hozzá tartozó tételeket (order_items tábla). Az adatintegritás megőrzése érdekében ezeket a lépéseket adatbázis tranzakciókba (DB::transaction) csomagoltuk. Ez biztosítja az "all-or-nothing" (minden vagy semmi) elvet: ha a rendelési tételek mentése közben bármilyen szerverhiba lépne fel, a rendszer automatikusan visszavonja (rollback) a fő rendelés létrehozását is, megelőzve az "árva", tételek nélküli rendelések kialakulását az adatbázisban.

Lekérdezések optimalizálása (Eager Loading) Az adminisztrátori felületen a rendelések és a hozzájuk tartozó termékek listázásakor kiemelt figyelmet fordítottunk az úgynevezett "N+1 lekérdezési probléma" elkerülésére. Ahelyett, hogy a rendszer minden egyes rendelési tételnél külön-külön futtatna egy adatbázis-lekérdezést a termék adatainak (pl. név, kategória) kinyerésére, a Laravel Eloquent ORM "Eager Loading" (with('items.product')) funkcióját alkalmaztuk. Ez a megoldás egyetlen, optimalizált SQL lekérdezéssel kéri le a kapcsolódó adatokat, minimalizálva a szerver terhelését és a válaszidőt.

3.5 API dokumentáció

Az API JSON formátumú válaszokat ad, a védett végpontok Sanctum tokenes hitelesítést igényelnek. A rendszer szabványos RESTful végpontokat biztosít a termékek, rendelések és felhasználói hitelesítés kezelésére. A válaszok felépítése egységes, ami megkönnyíti a kliensoldali feldolgozást.

API végpontok részletezése

Hitelesítési végpontok

Metódus	Útvonal	Leírás	Hitelesítés
POST	/api/register	Új felhasználó regisztrálása a rendszerbe.	Nem
POST	/api/login	Bejelentkezés és API token generálása.	Nem
POST	/api/logout	Kijelentkezés, az aktuális token törlése.	Sanctum

Termékkezelési végpontok

Metódus	Útvonal	Leírás	Hitelesítés
GET	**/api/products**	**Az összes elérhető termék listázása.**	**Nem**
POST	**/api/admin/products**	**Új termék létrehozása és képfeltöltés.**	**Admin**
DELETE	**/api/admin/products/{id}**	**Termék végleges törlése az adatbázisból.**	**Admin**

Rendeléskezelési végpontok

Metódus	Útvonal	Leírás	Hitelesítés
POST	/api/orders	Új rendelés létrehozása a kosár alapján.	User + Sanctum
GET	/api/my-orders	A bejelentkezett vásárló saját rendelései.	User + Sanctum
GET	/api/admin/orders	Összes beérkezett rendelés lekérése.	Admin + Sanctum
PUT	/api/admin/orders/{id}/status	Rendelés állapotának frissítése.	Admin + Sanctum

3.6 Tesztelési dokumentáció

A tesztelés Postman szoftverrel és manuális felületi teszteléssel történt. A folyamat célja annak igazolása volt, hogy a funkciók az elvárásoknak megfelelően működnek, és a jogosultságkezelés minden szinten érvényesül.

Postman tesztek: Minden API-végpontot ellenőriztünk Postman környezetben, szimulálva a sikeres és a hibás kéréseket is. Ellenőriztük a hitelesítési folyamatokat, a terméklistázást és az adminisztrátori műveleteket.

Frontend tesztek: A felület tesztelése manuálisan történt különböző böngészőkben, admin és vásárló szerepkörökkel. Vizsgáltuk a regisztrációt, a kosár működését és a rendelésleadási folyamatot.

A tesztelés módszertana A REST API fejlesztése során az API-First megközelítést alkalmaztuk. Mielőtt a Vue.js frontend elkészült volna, a backend végpontokat dedikált szoftveres környezetben, a Postman API kliens segítségével validáltuk. A tesztek során nemcsak a „Happy Path” (hibátlan működési útvonal) ellenőrzése történt meg, hanem kiemelt fókuszt kaptak a peremesetek (edge cases) és a hibakezelés (Error Handling) is. Ennek keretében vizsgáltuk az HTTP státuszkódok helyes visszatérését:

- **200 OK / 201 Created:** Sikeres adatlekérés, illetve új rekord (pl. rendelés, termék) sikeres létrejötte esetén.
- **401 Unauthorized / 403 Forbidden:** Jogosulatlan hozzáférési kísérletek esetén (pl. normál felhasználó próbál adminisztrátori végpontot hívni).
- **422 Unprocessable Content:** Hiányos vagy a validációs szabályoknak (pl. kötelező mezők, formátumok) nem megfelelő adatok beküldése esetén. A backend ilyenkor nemcsak elutasítja a kérést, hanem egy részletes JSON hibaobjektumot is visszaad, amelyből a frontend pontosan tudja tájékoztatni a felhasználót a hiba okáról.

A tesztelés során készült képernyőfotók igazolják a rendszer helyes működését és a hibaágak kezelését.

Postman API tesztelés:

kezmuves_tests > New Request

Save | Share

POST http://127.0.0.1:8000/api/login Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
{
    "email": "admin@webshop.hu",
    "password": "jelszo123"
}
```

Body 200 OK 436 ms 674 B Save Response

{} JSON Preview Visualize

```
{
    "success": true,
    "message": "Sikeres bejelentkezés!",
    "access_token": "2|kK1Js3xL35MVffEp7gyia7IFlfuvuSXe2kBmRd4tae00bca8",
    "user": {
        "id": 1,
        "name": "Főnök (Admin)",
```

kezmuves_tests > products

Save | Share

GET http://127.0.0.1:8000/api/products Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

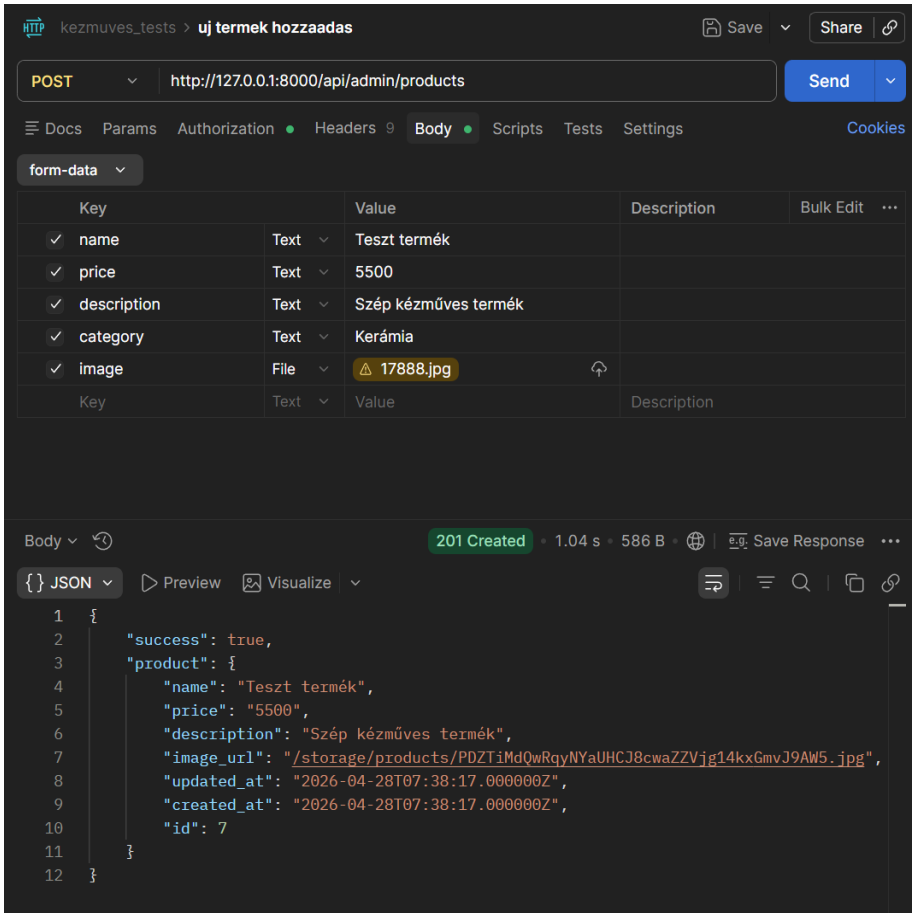
Key	Value	Description	Bulk Edit
Key	Value	Description	

Body 200 OK 206 ms 2.75 KB Save Response

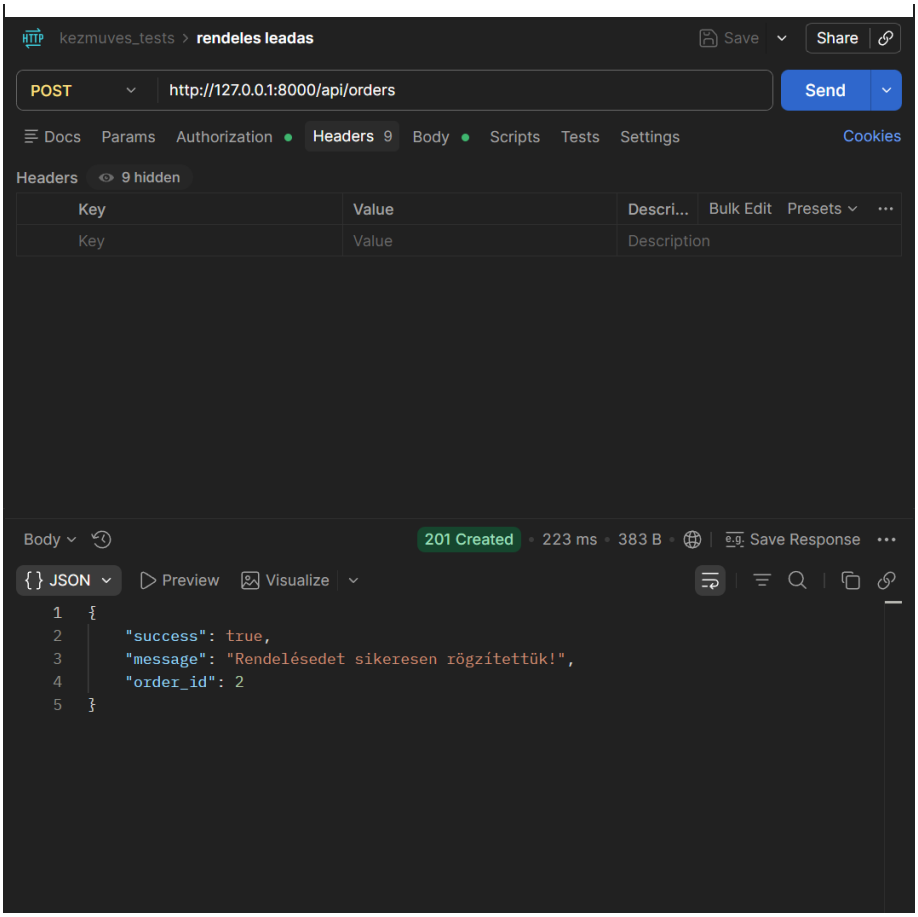
{} JSON Preview Visualize

```
[
    {
        "id": 1,
        "name": "Kék mázas kerámia bögre",
        "description": "Kézzel korongozott, egyedi kék mázzal égetett fél literes teásbögre. Mikrohullámú sütőben is használható.",
        "price": 4500,
        "category": "Kerámia",
        "image_url": "https://placehold.co/600x400/1e3a8a/ffffff?text=Kek+Mazas+Bogre",
        "created_at": "2026-04-28T07:05:14.000000Z",
        "updated_at": "2026-04-28T07:05:14.000000Z"
    },
    {
```

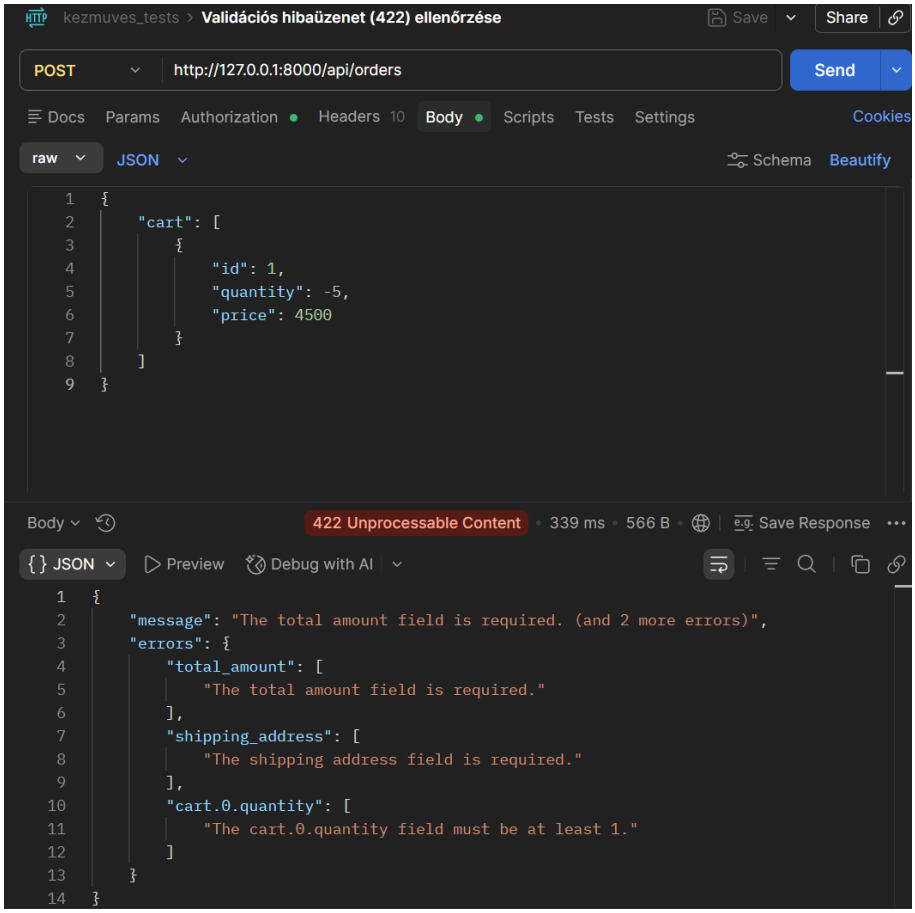
10. Ábra: Terméklista lekérdezése (GET /api/products)



11. Ábra: Új termék sikeres feltöltése admin kéréssel

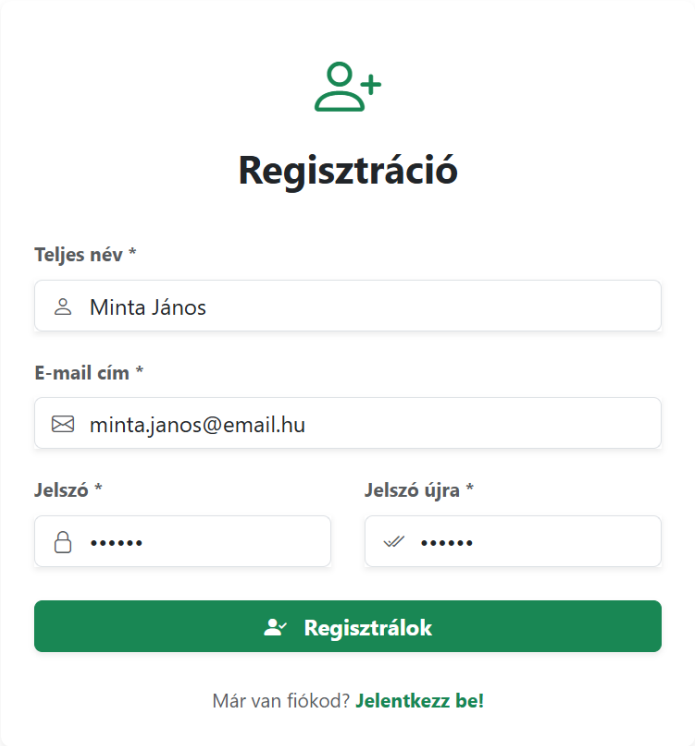


13. Ábra: Rendelés leadásának tesztelése JSON kéréssel

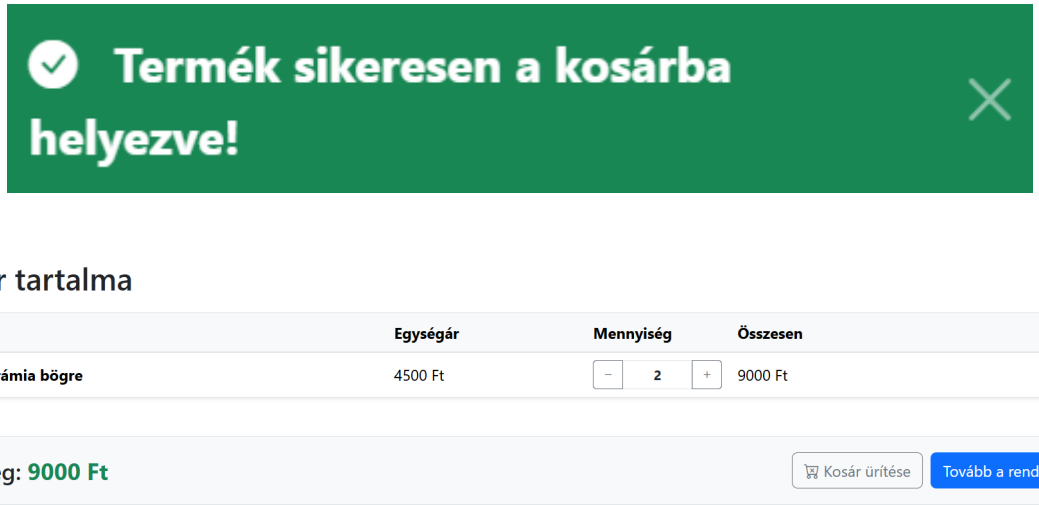


14. Ábra: Validációs hibaüzenet (422) ellenőrzése hiányos adatoknál

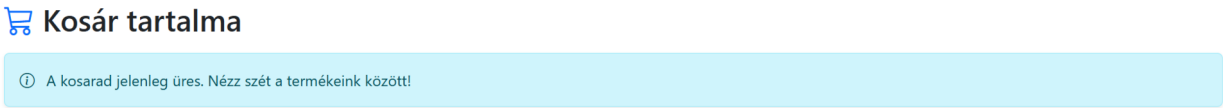
Manuális felületi tesztek:



15. Ábra: Regisztrációs folyamat tesztelése valós adatokkal

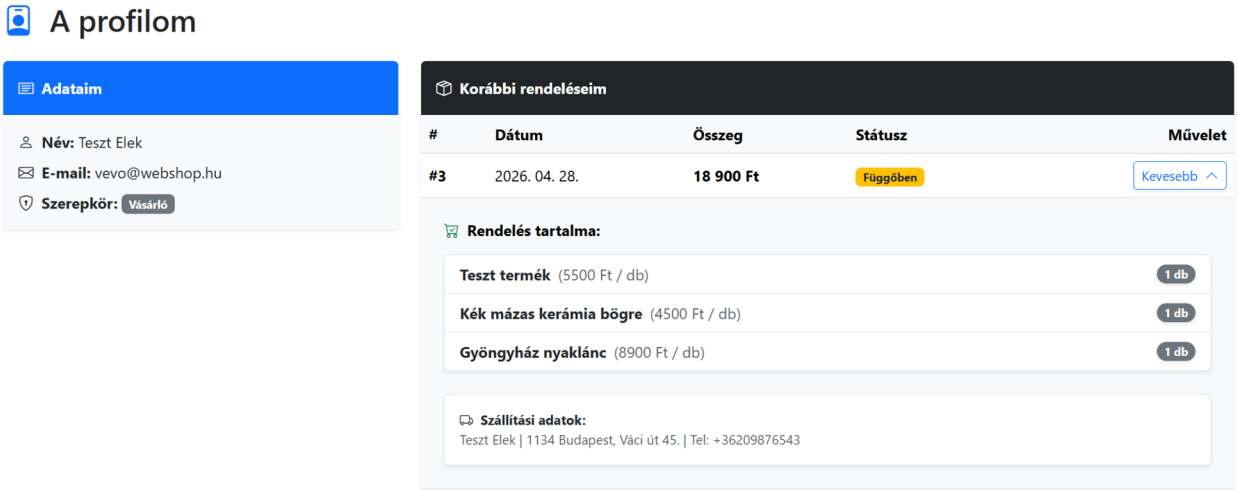


16. Ábra: Termék kosárba helyezése és a darabszám módosítása

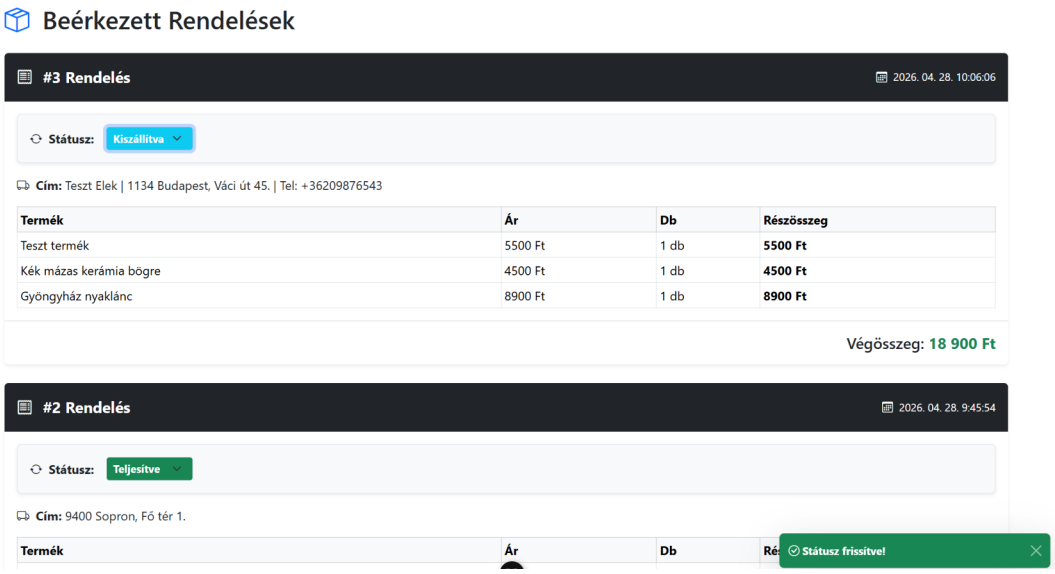


17. Ábra: Kosár ürítése

18. Ábra: Sikeres rendelésleadás utáni visszajelzés



19. Ábra: Profiloldal frissülése az új rendeléssel



20. Ábra: Adminisztrátori státuszmódosítás tesztelése

4 Továbbfejlesztési lehetőségek

A Kézműves Webshop jelenlegi verziója stabil alapokat nyújt az e-kereskedelmi tevékenységhez, de a piaci bevezetés előtt több funkcionális bővítés is javasolt. Az egyik legfontosabb fejlesztési irány a valós online fizetési megoldások (például Stripe, Barion vagy PayPal API) integrálása, amely kiváltaná a jelenlegi szimulált folyamatot.

Emellett a felhasználói élményt jelentősen javítaná egy automatizált e-mail értesítő rendszer kidolgozása a Laravel Mailable osztályai segítségével, amely a rendelések minden állapotváltozásáról tájékoztatná a vásárlót. Tervben van továbbá egy darabszám alapú raktárkészlet-kezelő modul, valamint egy vásárlói véleményező és csillagos értékelési rendszer implementálása is.

5 Összegzés

A projekt megvalósítása során egy modern, reszponzív és biztonságos webáruház alkalmazást hoztunk létre. A fejlesztés során sikerült a képzés alatt elsajátított elméleti és gyakorlati tudást – különösen a Laravel backend és a Vue.js frontend technológiákat – egy komplex rendszerben egyesíteni.

A csapatmunka során a feladatokat hatékonyan sikerült elosztani : a frontend kialakítása, a backend logika implementálása és a rendszerintegráció párhuzamosan, folyamatos egyeztetések mellett zajlott. A vizsgaremek fejlesztése nemcsak technikai tudásunkat mélyítette el, hanem rávilágított az agilis szoftverfejlesztési szemlélet és a precíz dokumentáció fontosságára is.

6 Források

- [1] **Laravel Keretrendszer** – Hivatalos dokumentáció (v11): https://laravel.com/docs
- [2] **Vue.js** – A progresszív JavaScript keretrendszer útmutatója: https://vuejs.org/
- [3] **Bootstrap 5** – Frontend komponens könyvtár dokumentáció: https://getbootstrap.com/
- [4] **MySQL** – Relációs adatbázis-kezelő kézikönyv: https://dev.mysql.com/doc/
- [5] **Axios** – Promise alapú HTTP kliens böngészőkhöz: https://axios-http.com/